



ADDING ORDER HISTORY TO YOUR EXPERIENCE

DRAFT

PDF



Last Updated: 10/25/2018

Retrieve a complete order history for your consumers.

TIP: Before using this guide you should have already completed [Adding Checkout to Your Experience](#).

Adding order history to your app is a two-step process.

- 1 Your application makes a BFF (backend-for-frontend) Order Summary API request to retrieve all or a filtered [list of a member's orders](#).
- 2 Using an order ID returned in the BFF Order Summary response, your application makes a BFF Order Details API request to [list order details for a member or guest](#).

What is an Order?

An order consists of all data related to:

- Consumer's purchased items/services
- Consumer's payment method(s) and billing address(es)
- Consumer's shipping method(s) and shipping address(es)
- Pricing
- Taxes

- Promotions
- Status

An order is created in the last step of [Checkout](#) when the consumer has provided all of the necessary checkout information and submits it for fulfillment. After an order is created, it is stamped with a unique order number, the date and time the order was submitted, and a status of `CREATED`. The order is assigned different statuses as it progresses through the order lifecycle. See [Understanding Order Status](#) for more detail.

Step 1: List a member's orders

Use the [BFF Order Summary API](#) to get all or a filtered list of orders for a Nike member. By making their past orders available to members as a self-service in your app, they can view their product and payment history without having to contact Consumer Services.

This API returns limited information about each order. If you need a more in-depth picture of an order that contains pricing, tax information, shipping information, and detailed product information, or if you want to list the details of a guest's order, see [List order details for a member or guest](#).

The BFF Order Summary API requires that you pass certain request headers. For more information, see [Required Request Headers](#).

Customizing Your Results

You control what is returned in your result set and how it is sorted through URL parameters.

Filtering

The table below lists the fields by which you can filter your BFF Order Summary results. If no filter is applied, all of a member's orders are returned. While some filters only allow one value, you can send multiple filters in the same request. For instance, even though only one `orderSubmitDateAfter` filter value is allowed, you can request a BFF Order Summary filtered by `orderSubmitDateAfter` and `status`. Note that filter parameter names and values are case sensitive.

Order Field Name	Description	Sample Value
status	List orders with this status.	Cancelled
orderType	List orders of this type. You can only filter by one orderType at a time.	SALES_ORDER
storeId	List orders placed in this store. You can only filter by one storeId at a time.	28382
orderSubmitDateAfter	List orders placed after this date. You can only filter by one date at a time.	Format is yyyy-MM-dd'T'HH:mm:ssZ

Sorting

You can sort a member's orders in several ways using the `sort` query parameter. You can sort by one or more order fields, separated by a comma. If the field name you want to sort by is nested, refer to it with dot notation. For sort parameter syntax, see the [Query Parameters](#) section of [Using NDe APIs](#).

TIP: It is recommended that your app pass the sort query parameter in the request to ensure that the results are sorted appropriately for your experience.

Other Query Parameters

The BFF Order Summary API also supports the `fields`, `count` and `anchor` query parameters to restrict the results to certain fields, restrict the number of results, and control pagination. For more information on syntax, see the [Query Parameters](#) section of [Using NDe APIs](#).

Let's take a look at some BFF Order Summary scenarios.

I want to list for a member	Sample Query
orderTypes of RESERVE_ORDER submitted after 2018-01-24 purchased at store 12345	<code>https://api.nike.com/ order_mgmt/user_order_summary/ v1?filter=storeId(12345)&filter =orderSubmitDateAfter(2018-01-2 4)&filter=orderType(RESERVE_ORD ER)</code>
orders with a status of Shipped or Delivered	<code>https://api.nike.com/ order_mgmt/user_order_summary/ v1?filter=status(Shipped,Delive red)</code>
all orders sorted in ascending modificationDate	<code>https://api.nike.com/ order_mgmt/user_order_summary/ v1?sort=modificationDateAsc</code>
just the order ID, status and submitted date fields of all orders	<code>https://api.nike.com/order_mgmt/ user_order_summary/ v1?fields=id,status,orderSubmitDate</code>
two most recently submitted orders	<code>https://api.nike.com/order_mgmt/ user_order_summary/v1?count=2</code>

Executing the Request

Sample CURL for BFF Order Summary for a Member/Employee

```
curl -X GET \
  https://api.nike.com/order_mgmt/user_order_summary/v1 \
  -H 'Authorization: Bearer
eyJhbGciOiJSUzI1NiIsImtpZCI6Ijc2YWIlbnThkLWMwZTMtNGVhYi05MTljLTJkYj
A3YjFjN2NhMHNpZyJ9.eyJpYXQiOiJlMzQxOTMyOTcsImV4cCI6MTUzNDE5Njg5Nyw
```

```
iaXNzIjoiY2IyOGE4OGItYWU4ZC00NWM1LWE2NjMtN
DRkMmY0NWQwZDZjIiwibGF0IjoxNTM0MTkzMjk3LCJhdWQiOiJjb20ubmlrZS5kaWd
pdGFsIiwic3ViIjoiY29tLm5pa2UuY29tbWVY2UubmlrZWRvdGNvbS53ZWl1LCJzY
nQiOiJuaWtlOmFwcCIsInNjcCI6WyJuaWtlLmRpZ2l0YWwiXSwicHJuIjoiMTYxODI
2OTIwMTIiLCJwcnQiOiJuaWtlOnN3b29zaCJ9.Nt-Irlborb2gmz6e-
CwUmvPlm80m5lEMHR18AEftE5qqVmlm-
HbFHNPPA6AWj8gscQRs02ft_CQtkvHza7EIvQ64RajD-
sj0FTTaPBMXUsWqL1JtlfFv61cYmbrErOsEBcV_NWbgOVQ_NNF3aL9FCLi12OgrVi1
pa7ManTLlOP_nmI_SaMN3USawECzKbYlOW58DaHBQjezkpeejyv4AQXm99HL1qWYb5
fARnpubrwlcN7GyUepOwImfNf8xZZrcJKTx4HBXmYVFg8gMskoqzSEjJijfxYNxSy
507Y4fUyRq2glISPMnrQnAF5NAwl9SCQm8wZxKxPaxc590EhMLA'
```

Parsing the Response

The BFF Order Summary JSON response contains several fields relating to status. See [Understanding Order Status](#) for more detail about how status is determined and the suggested order statuses to display in your experience. See the [BFF Order Summary API](#) for a full list of fields returned in the response.

Step 2: List order details for a member or guest

Use the [BFF Order Details API](#) to get order details for a member or guest. This API returns a complete picture of an order including product detail, tax information and line item details. If you are looking for higher level order information or you want information on more than one order for either a member or employee, see [List a member's orders](#).

TIP: The BFF Order Details API does not return image

URL but you can call the [Merchandised Product API](#) using the style-color returned from the BFF Order Details API to get a list of images for a styleColor and country.

The BFF Order Details API requires that you pass certain headers in the request depending upon whether the consumer is a member, guest, or employee. For more information, see [Required Request Headers](#).

Required Request Parameters

For members and employees, the orderNumber path parameter is required. For validation purposes for guest consumers, the orderNumber path parameter and email address filter parameter are required.

TIPS: To avoid a 404 response, the authentication header for a member request must match the member who created the order.

For guests, the email address must match the shipTo email address on the order.

Customizing Your Results

You control what is returned in your result set through URL parameters. The BFF Order Details API supports the `fields` query parameter to restrict the fields returned in the response. Since the BFF Order Details API only returns one consumer order, the `anchor`, `sort`, `filter` and `count` query parameters are not supported. For more information on the `fields` query parameter syntax, see the [Query Parameters](#) section of [Using NDe APIs](#).

Let's take a look at some BFF Order Details scenarios.

I want to	Sample Query
list the details for order ID C00011554850 for a member	<code>https://api.nike.com/order_mgmt/user_order_detail/v1/C00011554850</code>
list the ID, status and shipping method fields for orderNumber C00011554850 for a guest	<code>https://api.nike.com/order_mgmt/user_order_details/v1/C00011554850?filter=email(my.email@address.com)&fields=id,status,orderLines.shippingMethod</code>

Executing the Request

Sample CURL for BFF Order Details C00011554850 for a member/employee:

```
curl -X GET \
  https://api.nike.com/order_mgmt/user_order_detail/v1/
C00011554850 \
  -H 'Authorization: Bearer
eyJhbGciOiJSUzI1NiIsImtpZCI6IjY2YWI1NTlkLWwzMtNGVhYi05MTljLTJkYjA3YjFjN2NhMHNpZyJ9.eyJpYXQiOiJlMzQxOTMyOTcsImV4cCI6MTUzNDE5Njg5NywiaXNzIjoib2FldGgyYWNjIiwianRpIjojY2IyOGE4OGItYWU4ZC00NWw1LWE2NjMtNDRkMmY0NWQwZDZjIiwibGF0IjoxNTM0MTkzMjk3LCJhdWQiOiJjb20ubmlrZS5kaWdpdGFsIiwic3ViIjojY29tLm5pa2UuY29tbWVhY2UubmlrZWVdGNvbS53ZWl1LCJzYnQiOiJuaWtlOmFwcCI6WyJuaWtlLmRlZ2l0YWwixSwicHJlIjojMTYxODI2OTIwMTIiLCJwcnQiOiJuaWtlOnN3b29zaCJ9.Nt-Irlborb2gmz6e-CwUmvPlm80m51EMHR18AEftE5qqVmlm-HbFHNPPA6Awj8gscQRs02ft_CQTkvHZA7EivQ64RajD-sj0FTTaPBmXUsWqL1JtlfFv61cYmbrErOsEBcV_NWbgOVQ_NNF3aL9FCLi12OgrVi1pa7ManTL1OP_nmI_SaMN3USawECzKbYlOW58DaHBQjezkpeejyv4AQXm99HL1qWYb5fARnpuBrwlcN7GyUepOwImfNf8xZZrcJKTx4HBXmYVFg8gMskoqzSEjJijfxYNxSy
```

507Y4fUyRq2glISPMnrQnAF5NAw19SCQm8wZxKxPaxc59OEhMLA'

Sample CURL for BFF Order Details C00011554850 for a guest:

```
curl -X GET \
  'https://api.nike.com/order_mgmt/user_order_details/v1/
  C00011554850?filter=email%28my.email@address.com%29' \
  -H 'X-Nike-Visitorid: 2c83877b-10fa-44da-a92d-2451efef8671' \
  -H 'appid: com.nike.sport.running.ios' \
  -H 'x-nike-visitid: 2'
```

Parsing the Response

The BFF Order Details JSON response contains several fields relating to status. See [Understanding Order Status](#) for more detail about how status is determined and the suggested order statuses to display in your experience. See the [BFF Order Details API](#) for a full list of fields returned in the response.

Understanding Order Status

An order contains three types of statuses:

- order
- order line
- payment

An order line is a Nike service or product associated with a quantity, e.g. Nike Air VaporMax quantity 1. Orders have at least one order line and may have several. Each order line has one or more statuses that map(s) to a numeric status code. Behind the scenes, the status of the order is calculated by evaluating which order line has a rolledUpStatus with the highest status code.

The table below describes each status associated with an order.

Status Field Name	Description	API
status	Status of the order. Matches the orderLines.rolledUpStatus with the highest status code on the order.	BFF Order Summary BFF Order Details
orderLines.rolledUpStatus	Status of an order line. Computed by “Partially” + orderLines.maxOrderLineS tatus. e.g. “Partially Shipped”.	BFF Order Summary BFF Order Details
orderLines.maxOrderLineS tatus	Status of the highest status code on this order line e.g “Shipped”. Matches orderLines.rolledUpStatus. This status is included in orderLines.statuses.	BFF Order Details
orderLines.minOrderLineSt atus	Status of the lowest status code on this order line e.g. “Factory Delayed”. This status is included in orderLines.statuses.	BFF Order Details
orderLines.statuses	An array of statuses for each status code on this order line. The description with the highest status code matches orderLines.maxOrderLineS tatus. The description with the lowest status code matches orderLines.minOrderLineSt	BFF Order Details

Status Field Name	Description	API
	atus.	
paymentStatus	Status of payment.	BFF Order Summary

In the scenario illustrated by the BFF Order Details response below, a consumer has purchased two order lines. One order line has shipped and has a rolledUpStatus of “Shipped” and the other has been delayed at the factory and has a rolledUpStatus of “Factory Delayed”. Because the “Shipped” status has a higher status code than the “Factory Delayed” status code, the order status is “Partially Shipped”, calculated by “Partially” + the order line with the highest rolledUpStatus on the order.

```
{
  "status": "Partially Shipped",
  ...
  "orderLines": [
    {
      ...

      "statuses": [
        {
          "description": "Shipped",
          "quantity": 1,
          "date": "2018-01-30T12:16:51Z"
        }
      ],
      "rolledUpStatus": "Shipped",
      ...
      "maxOrderLineStatus": "Shipped",
      ...
    }
  ]
}
```

```

        "minOrderLineStatus": "Shipped",
        ...
    },

```

Let's look at a slightly more complex example. As illustrated by the BFF Order Details response below, a consumer purchased three identical shorts. One short was delivered and has a status of "Delivered", another short was delivered and returned and has a status of "Return Processed", and one short was shipped and has a status of "Shipped". Because the "Delivered" status has the highest status code of the order line, the rolledUpStatus of the order line is "Delivered". The highest rolledUpStatus of the order is "Delivered", so the order status is "Partially Delivered".

```

        "quantity": 1,
        "date": "2018-01-30T12:16:51Z"
    }
}
"status": "Partially Delivered",
...
"rolledUpStatus": "Factory Delayed",
"orderLines": [
    {
        "maxOrderLineStatus": "Factory Delayed",
        ...
        "minOrderLineStatus": "Factory Delayed",
        "statuses": [
            {
                "description": "Delivered",
                "quantity": 1,
                "date": "2018-01-30T12:16:51Z"
            },
            {
                "description": "Shipped",
                "quantity": 1,
                "date": "2018-01-30T12:16:51Z"
            }
        ]
    }
]
}

```

```

    "description": "Return Processed",
    "quantity": 1,

```

Listed below are the order line statuses, status codes, and simple status.

STATUS: The order line status is returned by both the BFF Order Summary and BFF Order Details APIs.

STATUS CODE: Behind the scenes, status code is used by both APIs to calculate status ranking. However, the status code is not returned by either the BFF Order Summary or BFF Order Details API.

SIMPLE STATUS: The simple status is not returned by either the BFF Order Summary or BFF Order Details API but is provided as a sample, consumer-friendly status. Your experience could perform a similar status-to-simple-status mapping to display status to the consumer.

STATUS	STATUS CODE * not returned by either API	SIMPLE STATUS * not returned by either API
Draft Order Created	1000	CREATED
Draft Order Reserved Awaiting Acceptance	1040	CREATED
Draft Order Reserved	1050	CREATED
Created or CREATED	1100	CREATED
Authorized	1100.01	CREATED
Not Authorized	1100.02	CREATED
Pending Completion	1100.03	CREATED
Origin Scan	1100.050	CREATED
In Transit	1100.100	CREATED
Acknowledged	1100.110	IN PROCESS

STATUS	STATUS CODE * not returned by either API	SIMPLE STATUS * not returned by either API
Work Order Cancelled	1100.1600	IN PROCESS
Factory Processing	1100.600	IN PROCESS
Factory Completed	1100.700	IN PROCESS
Factory Delayed	1100.800	IN PROCESS
Reserved Awaiting Acceptance	1140	IN PROCESS
Reserved	1200	IN PROCESS
Being Negotiated	1230	IN PROCESS
Accepted	1260	IN PROCESS
Rescheduling	1300	IN PROCESS
BOReview	1300	IN PROCESS
Unscheduled	1310	IN PROCESS
Scheduled	1500	IN PROCESS
Awaiting Chained Order Creation	1600	IN PROCESS
Awaiting Procurement Purchase Order Creation	2030	IN PROCESS
Awaiting Procurement Transfer Order Creation	2060	IN PROCESS
Chained Order Created	2100	IN PROCESS
PO Released	2100.100	IN PROCESS
Released For SO Create	2100.1000	IN PROCESS

STATUS	STATUS CODE * not returned by either API	SIMPLE STATUS * not returned by either API
DSV Acknowledged	2100.1100	IN PROCESS
Released To Third Party	2100.1200	IN PROCESS
Received By Partner	2100.1210	IN PROCESS
Confirmed By Partner	2100.1220	IN PROCESS
Order Processing	2100.1230	IN PROCESS
Order Delayed	2100.1250	IN PROCESS
PO Acknowledged	2100.200	IN PROCESS
Released To FC	2100.30	IN PROCESS
PO Sent To Warehouse	2100.300	IN PROCESS
PO Warehouse Acknowledged	2100.350	IN PROCESS
PO Scheduled For Pick	2100.40	IN PROCESS
Order Received	2100.400	IN PROCESS
PO Awaiting Shipment	2100.50	IN PROCESS
Order Processed	2100.500	IN PROCESS
DSV Acknowledged	2100.550	IN PROCESS
Factory Processing	2100.600	IN PROCESS
Sent For SO Create	2100.620	IN PROCESS
DSV Acknowledged	2100.650	IN PROCESS
Factory Completed	2100.700	IN PROCESS
Factory Delayed	2100.800	IN PROCESS

STATUS	STATUS CODE * not returned by either API	SIMPLE STATUS * not returned by either API
Procurement Purchase Order Created	2130	IN PROCESS
Procurement Purchase Order Shipped	2130.01	IN PROCESS
Work Order Created	2140	IN PROCESS
Work Order Completed	2141	IN PROCESS
Procurement Transfer Order Created	2160	IN PROCESS
Procurement Transfer Order Shipped	2160.01	IN PROCESS
Released	3200	IN PROCESS
Awaiting Shipment Consolidation	3200.01	IN PROCESS
Awaiting WMS Interface	3200.02	IN PROCESS
Published To WMS	3200.03	IN PROCESS
Acknowledged	3200.100	IN PROCESS
Sent To Warehouse	3200.1000	IN PROCESS
Released To Third Party	3200.1200	IN PROCESS
Received By Partner	3200.1210	IN PROCESS
Confirmed By Partner	3200.1220	IN PROCESS
Order Processing	3200.1230	IN PROCESS
Pending Release To SAP	3200.150	IN PROCESS
Scheduled For Pick	3200.200	IN PROCESS

STATUS	STATUS CODE * not returned by either API	SIMPLE STATUS * not returned by either API
Awaiting Shipment	3200.300	IN PROCESS
Order Received	3200.400	IN PROCESS
Order Processed	3200.500	IN PROCESS
Factory Processing	3200.600	IN PROCESS
Factory Completed	3200.700	IN PROCESS
DSV Acknowledged	3200.900	IN PROCESS
Sent To Node	3300	IN PROCESS
Included In Shipment	3350	IN PROCESS
Awaiting Invoice	3700.0101	IN PROCESS
Invoiced	3700.999	IN PROCESS
Held	8000	IN PROCESS
Carried	1100.7777	COMPLETE
Order Completed	2100.1240	COMPLETED
Order Completed	3200.1240	COMPLETED
Completed	3700.12	COMPLETED
Order Completed	3700.777710	COMPLETED
Order Completed	3700.7777.10	COMPLETED
Backordered From Node	1400	BACKORDER
Order Delayed	3200.1250	DELAYED
Factory Delayed	3200.800	DELAYED
Shipped	3700	SHIPPED

STATUS	STATUS CODE * not returned by either API	SIMPLE STATUS * not returned by either API
Origin Scan	3700.10	SHIPPED
Customs Cleared	3700.100	SHIPPED
Manifest Scan	3700.200	SHIPPED
In Transit	3700.30	SHIPPED
Delivered To Store	3700.105	DELIVERED
Notionally Delivered	3700.69	DELIVERED
Delivered	3700.70	DELIVERED
Order Delivered	3700.7777	DELIVERED
Ready For Pickup	3700.110	READY FOR PICKUP
Customer Picked Up	3700.120	PICKED UP
Picked Up	3700.20	PICKED UP
Shipment Delayed	3700.130	SHIPMENT DELAYED
Shipment Delayed	8500	SHIPMENT DELAYED
Abandoned	3700.210	ABANDONED
Delivery Attempt 1	3700.40	DELIVERY ATTEMPT 1
Delivery Attempt 2	3700.50	DELIVERY ATTEMPT 2
Delivery Attempt 3	3700.60	DELIVERY ATTEMPT 3
Unable To Deliver	3700.80	UNABLE TO DELIVER
Refused Shipment	3700.90	REFUSED SHIPMENT
Pending Cancel	1300.9000	CANCEL IN PROCESS
Pending Cancel	3200.9000	CANCEL IN PROCESS

STATUS	STATUS CODE * not returned by either API	SIMPLE STATUS * not returned by either API
Cancelled	9000	CANCELLED
Void	9000.100	CANCELLED
Post Voided	9000.200	POST VOIDED
Delivered to Return Center	1100.200	RETURN CREATED
Early Refund	1100.300	RETURN CREATED
Return Created	3700.01	RETURN CREATED
Return Request Created	3700.011	RETURN CREATED
Return Received	3700.02	RETURN CREATED
Returned At Store	1100.400	RETURN COMPLETED
Returned At Store	3700.04	RETURN COMPLETED
Returned to DC	3700.85	RETURN COMPLETED
Returned To Carrier	3700.95	RETURN COMPLETED
Delivered to Return Center	1100.200100	RETURN IN PROCESS
Inspection Escalate	1100.500	RETURN IN PROCESS
Return In Progress	3700.03	RETURN IN PROCESS
Return Processed	3700.05	RETURN IN PROCESS
Return Invoiced	3950.01	RETURN IN PROCESS
Received at Return Center	3950.100	RETURN IN PROCESS
Inspection Passed	3950.200	RETURN IN PROCESS
Inspection Escalate	3950.300	RETURN IN PROCESS
Inspection Denied	3950.400	RETURN IN PROCESS

STATUS	STATUS CODE * not returned by either API	SIMPLE STATUS * not returned by either API
Inspected And Invoiced	3950.01.100	RETURN IN PROCESS
Return Denied	3700.06	RETURN DENIED
CSR Refund	3950.700	CSR REFUND
Inspected And Invoiced	3950.01100	RETURN PROCESSED
Exchange Order Created	3950.02.01	EXCHANGE CREATED
Awaiting Exchange Order Creation	3950.02	EXCHANGE IN PROCESS
CREATED		Created (Only for Reserve Orders)
UNRESERVED		UnReserved(Only for Reserve Orders)

API Endpoint Quick Reference

Endpoint Name	Path	HTTP Method
BFF ORDER SUMMARY	/order_mgmt/ user_order_summary/v1	GET
BFF ORDER DETAIL	/order_mgmt/ user_order_detail/ v1/{orderNumber}	GET

Best Practices

Listed below are some best practices for working with BFF Order Summary and BFF Order

Details.

Conditions for Retries

For all Nike Cloud APIs, the general rule is that HTTP 4XX error codes (except for 429) should not be retried but HTTP 5XX errors can be retried. For general information on Nike error retry practices, see [API Error Patterns](#) on Confluence.

Test Environment

It is recommended to test all Order endpoints in the production environment as opposed to the test environment. Using the test environment can have unpredictable results due to the many downstream services which these endpoints are reliant upon in order to provide typical 'production-like' responses.

There are boundaries for testing in production:

- Performance tests at high volumes should never be done in production. All performance tests should be done in test.

Caching Data

None of the endpoints described in this document support caching.

Troubleshooting

- Use the general troubleshooting tips in the [Using NDe APIs](#) guide.
- Use a Splunk query (requires access) to check for issues with your request.
- Contact the Orders team on the [#mp-athena](#) Slack channel for assistance.

Terms of Service

Authorization

Access Tokens

Most calls through the Nike API gateway (api.nike.com) require an access token be sent in the request header. This allows Nike to verify that your app is authorized to perform the action on behalf of the user.

Access tokens are obtained by calling Nike Unite services prior to calling the API which you ultimately want to reach.

To find out more on how to call Unite services to obtain access tokens, see the Authorization section of the [Using NDe APIs](#) guide.

JSON Web Token

Neither the BFF Order Summary nor BFF Order Details endpoints require the additional authorization of a JSON Web Token (JWT). For more, see the JWT section of [Using NDe APIs](#).

Sample Requests

Sample requests included throughout this guide contain unique IDs and access tokens that are spent/expired in the Production environment, so you will not be able use them as-is for testing purposes. Reuse what you can and replace with valid IDs/access tokens when necessary.

User Types

The BFF Order APIs support 3 distinct user types:

- Member: user has logged in with their Nike+ account credentials
- Guest: user has not logged in (anonymous user)

- Employee: user is an employee of Nike or a subsidiary and has logged in with swoosh.com credentials (also known as Swoosh user type)

Required Request Headers

Listed below are the required request headers based on user type. Since most BFF Order Summary and BFF Order Details requests come through the Nike Edge router, these header values will be set automatically, provided your app experience calls the Unite services first to get an access token and passes that token in the request.

Header Name	Description	Member	Guest * guest consumers are supported in the BFF Order Details API only	Employee
Accept	Content type you will accept in response, application/json is only value allowed	X	X	X
Content-Type	Content type of the request, application/json is only value allowed	X	X	X
Authorization	Your access token in the format of Bearer {token} indicating the	X	X	X

Header Name	Description	Member	Guest * guest consumers are supported in the BFF Order Details API only	Employee
	consumer is logged in			
x-nike-visitorid	Unique identifier for the guest, validated by the Edge router and passed through to the service. Applies only to BFF Order Details API.		X	
x-nike-visitid	Integer identifying the guest's session. Applies only to BFF Order Details API.		X	
appld	Application making the API request e.g. com.nike.sport.running.ios	X	X	X

TIP: For the Authorization header, use the token for the consumer's login session that you obtained from Nike Unite/Identity, prefixed by ****Bearer **** (note the single space after Bearer). This is necessary for Nike to verify that your app is authorized to perform the requested operation on behalf of the consumer.

See the User Types section of the [Using NDe APIs](#) guide for more information.

Common Questions

Is it okay to call Order APIs if my app is hosted in an Amazon Web Services VPC?

Yes. The APIs are exposed publicly so it does not matter where you are calling from. If you are calling repeatedly from a small set of IP addresses, it might be possible that Nike's bot-mitigation tools could interfere with your ability to make calls. If you are having issues, reach out to Slack channel [#mp-athena](#) for help.

Contacting the Team

Need to contact the Orders team?

Slack	#mp-athena
Confluence Space	Order Management
Team Contacts	Intake, new requirements, onboarding Betty Ashok Betty.Ashok@nike.com Lindsey Kiken Lindsey.Kiken@nike.com Krishnamurthy Ramakrishnan Krishnamurthy.Ramakrishnan@nike.com API or service-related issues

	Vishibha Anand Vishibha.Anand@nike.com
Intake	Please fill out an intake form to initiate a requirement request. For more information, see the Marketplace Platform Intake Process .

Glossary

See the [Glossary](#) for related terms.

Document Change Log

Summary	Date	Description
Initial draft 10/25/2018	Initial Draft	

Next Steps

You've learned how to add Order History to your experience. Here are some next steps.

- [Capturing User Events](#)
- [Using NDe APIs](#)